

DALI · DESIGN ENGINEERING 26F

Design Engineering

Session 1: Agentic Coding & Git

Petros Taxiarchis · September 29, 2026

4:00–6:00 PM · Room B05

BEFORE WE START

Join the class GroupMe



Scan while we wait for everyone to settle in.

TODAY, AT A GLANCE

The whole run of show

4:00	Hook
4:03	Welcome & course overview
4:11	Icebreaker (whole class)
4:21	Install, together
4:35	Buffer
4:37	Git & GitHub, live demo
4:53	Terminal basics
4:59	Clone & run
5:07	Claude Code + meet the API
5:14	Break + form teams
5:19	Team icebreaker
5:24	Team activity: explore + design
5:48	Show & tell
5:57	Wrap-up

HOOK

4:00–4:03 · 3 min

HOOK

Before anything else...

- It's already running: `api-backend-zeta-five.vercel.app`
- No frontend yet. That's what your team decides on today.
- "What would you build on this?" Take 2–3 shouted answers.

WELCOME

4:03–4:05 · 2 min

LECTURE

What this course actually is

Software engineering with coding agents: not "typing code," and not "let the AI do it and hope."

Design isn't a separate track here: it's handled the way engineers handle it, by loading the right skill (impeccable, Emil Kowalski's design skill, svg-animations) and judging what comes out.

WELCOME

4:05–4:06 · 1 min

LECTURE

What you'll learn, part 1

- Choose a stack, and defend the choice
- Design and test APIs (Bruno)
- Model data and write real SQL
- Manage an agent's context on purpose

WELCOME

4:06–4:07 · 1 min

LECTURE

What you'll learn, part 2

- Extend the agent with skills and MCP servers
- Run multi-agent workflows
- Ship something real

WELCOME

4:07–4:09 · 2 min

LECTURE

Where we're headed: six sessions

- 01 Agentic Coding & Git **today**
- 02 Stack Choices & System Architecture
- 03 Context Engineering & Skills
- 04 APIs, HTTP & Bruno (+ MCP)
- 05 Databases & SQL
- 06 Multi-Agent Workflows, Verification & Ship

WELCOME

4:09–4:11 · 2 min

LECTURE

Course expectations

Format

- 6 sessions × 2 hours
- Lecture + live demo
- Hands-on lab
- Show & tell

To get the certificate

- At most 2 absences
- Present in Session 6

ICEBREAKER

4:11–4:13 · 2 min

ICEBREAKER

Icebreaker: meet the room

- Quick-fire round the room: one sentence per prompt, about 20 seconds each
- ~24 of us, ~10 minutes total, so keep it snappy
- Next slide has exactly what to say

ICEBREAKER

4:13–4:21 · 8 min

ICEBREAKER

Icebreaker: your turn

1. Your name
2. Your class year
3. Why you took this class / what you want to learn
4. Your experience with CS, web dev, or design: none, a little, or a lot

That's it, one breath, then the next person goes.

LET'S INSTALL

4:21–4:35 · 14 min

HANDS-ON

Let's install, together, right now

Install today

- Node.js (LTS)
- git
- VS Code or Cursor
- Claude Code: install and sign in
- Bruno
- A GitHub account

Not today

- Postgres
- A database GUI

Our shared prototype has no database yet. It's in-memory data, and those come back for real in Session 5.

BUFFER

4:35–4:37 · 2 min

BUFFER

Buffer

Installs almost always run long. Use this for stragglers, or skip straight ahead if everyone's ready.

FOUNDATIONS

4:37–4:39 · 2 min

LECTURE

What is Git & GitHub?

- **Git**: tracks changes to your code over time, on your own machine
- **GitHub**: hosts git repositories online, so people (and agents) can collaborate on them
- Core idea: a **commit** is a saved checkpoint you can always get back to
- Why it matters here: git is the safety net that makes an agent's mistakes reversible

FOUNDATIONS

4:39–4:43 · 4 min

LIVE DEMO

Live demo, part 1: branch, commit, break it

1. Create a branch
2. Have the agent add a feature
3. Commit in small steps
4. Let the agent break something, on purpose

FOUNDATIONS

4:43–4:47 · 4 min

LIVE DEMO

Live demo, part 2: restore, PR, merge

1. `git restore` / `git revert` to recover
2. Re-prompt with a better spec
3. Open a pull request
4. Merge it

FOUNDATIONS

4:47–4:51 · 4 min

LECTURE

Git & GitHub: commands you'll actually use

- `git status` : what changed?
- `git add .` : stage those changes
- `git commit -m "message"` : save a checkpoint
- `git push` / `git pull` : sync with GitHub
- Branch → change → commit → open a pull request → merge

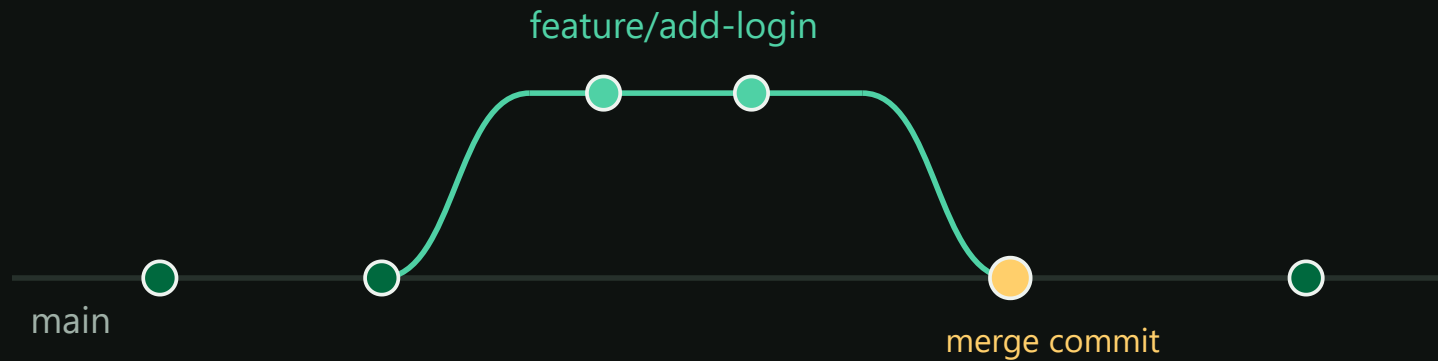
Rule of thumb we'll repeat all course: **commit before you let an agent touch anything.**

FOUNDATIONS

4:51–4:53 · 2 min

LECTURE

How branching & merging works



FOUNDATIONS

4:53–4:59 · 6 min

LECTURE

Terminal basics

- `pwd` : where am I?
- `ls` : what's here?
- `cd folder-name` : go into a folder (`cd ..` goes up)
- `mkdir name` : make a folder
- `cat file` : print a file's contents
- Tab-complete file and folder names, don't type them out by hand
- Environment variables: settings your program reads from outside the code (e.g. secrets, ports)

FOUNDATIONS

4:59–5:02 · 3 min

LECTURE

Cloning a project

- `git clone <repo-url>`: downloads a full copy of a repo, history included, into a new folder
- GitHub gives you that URL from the green "Code" button on any repo page
- `cd` into the new folder once it's done
- From there it's a normal local git repo: commit, branch, push, pull, all of it

FOUNDATIONS

5:02–5:07 · 5 min

HANDS-ON

Clone & run this project

- `git clone https://github.com/taxpet06/design-engineering-26f-api.git`
- `cd design-engineering-26f-api`
- `npm install`: downloads its dependencies
- `npm run dev`: starts the server locally
- Open `http://localhost:3000` in a browser: same API, running on your machine

FOUNDATIONS

5:07–5:10 · 3 min

HANDS-ON

Setting up Claude Code

- Install the CLI, then sign in with your account
- It asks before it edits or runs anything: you stay in control (permission modes)
- Today: just get it installed and signed in
- We'll go deep on how it actually works in later sessions

MEET THE PROJECT

5:10–5:12 · 2 min

LECTURE

The live API

- Live API: `api-backend-zeta-five.vercel.app`
- GitHub repo: `github.com/taxpet06/design-engineering-26f-api`
- No frontend yet, and no real database: in-memory data, regenerated fresh on every restart

MEET THE PROJECT

5:12–5:14 · 2 min

LECTURE

What data it has

- Academics: students, courses, sections, enrollments, grades, assignments
- Campus life: clubs, events, dining, housing, gyms, intramurals
- Extras: workshops, wellness logs, scholarships, research, jobs, lost & found

TEAM FORMATION

5:14–5:19 · 5 min

BREAK

Stand up. Stretch. Form your team.

- Groups of about 3–4
- Try to end up with people whose icebreaker answers you don't already know

ICEBREAKER

5:19–5:24 · 5 min

ICEBREAKER

Team icebreaker: summer fun facts

Go around the team, about 30 seconds each:

- Share one fun fact about your summer

Name your team after your favorite fact you just heard. You'll use that name in show & tell.

IN-CLASS ACTIVITY

5:24–5:31 · 7 min

TEAM ACTIVITY

Explore, in your team

- Open the live URL and click around a few endpoints: try `/students`, `/courses/:id/rating`, `/students/:id/profile`
- Skim the GitHub repo, and read the README to see what each file does
- Discuss: who would actually use a page built on this data, and what's the one thing they came here to do?

IN-CLASS ACTIVITY

5:31–5:36 · 5 min

TEAM ACTIVITY

Guidelines, part 1: scope it

- Pick **one page** of the eventual class website, not the whole site
- You don't have to use all the data available: pick only what your page actually needs
- Name the page's **one core user flow**, start to finish

IN-CLASS ACTIVITY

5:36–5:41 · 5 min

TEAM ACTIVITY

Guidelines, part 2: shape it

- List which API endpoints that flow would call
- Think through its states: empty, loading, error, too many / too few results
- Sketch it: paper, Figma, or just words. We'll build with agents in later sessions.

IN-CLASS ACTIVITY

5:41–5:48 · 7 min

TEAM ACTIVITY

Every team leaves with a one-pager

- Your team name
- A page name and who it's for
- Its one core user flow
- The endpoints that flow needs

This becomes the seed your team builds on for the rest of the course.

SHOW & TELL

5:48–5:57 · 9 min

SHOW & TELL

Show & tell

- Team name, page name, who it's for, and your one core flow
- ~60–90 seconds each

WRAP-UP

5:57–6:00 · 3 min

WRAP-UP

Next time

- Finish any installs you didn't get to
- Your team's one-pager carries into next session
- Session 2: Stack Choices & System Architecture

Thank you!

Please stay for a moment, we're doing attendance in person.